



# SHADOWCRYPT

## TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Threat Model
4. Cryptographic Design
5. Steganographic Design
  - 5.1 Baseline weakness Addressed
  - 5.2 Scattered, Password-Derived Embedding Order
  - 5.3 LSB Matching instead of LSB Replacement
6. Testing
7. Known Limitations
8. Possible Feature Work
9. Instruction For Use.
10. Conclusion

## List of Abbreviations

| Abbreviation | Expansion                                            |
|--------------|------------------------------------------------------|
| AES          | Advanced Encryption Standard                         |
| GCM          | Galois/Counter Mode                                  |
| AEAD         | Authenticated Encryption with Associated Data        |
| PBKDF2       | Password-Based Key Derivation Function 2             |
| HMAC         | Hash-based Message Authentication Code               |
| SHA          | Secure Hash Algorithm                                |
| LSB          | Least Significant Bit                                |
| RS Analysis  | Regular-Singular Analysis (a steganalysis technique) |
| PNG          | Portable Network Graphics                            |
| JPEG         | Joint Photographic Experts Group                     |
| DCT          | Discrete Cosine Transform                            |
| CLI          | Command-Line Interface                               |
| ML           | Machine Learning                                     |
| KDF          | Key Derivation Function                              |

# ShadowCrypt: Encrypted Image Steganography Tool

## 1. Abstract

ShadowCrypt is a command-line utility that integrates authenticated symmetric encryption (AES-256-GCM) and image-based steganography (LSB embedding) into one tool, allowing for hiding the encrypted message within a PNG file. The principle of the tool is defense in depth. Namely, while detection of hidden information inside the image may be possible, the message itself will be secured by the use of encryption. Moreover, the embedding technique used in ShadowCrypt is more resistant to steganalysis than the simple LSB method.

---

## 2. Introduction

Steganography is the practice of concealing the secret messages in other kinds of media like images, audios, and videos in such a way that they are not visible to the naked eye. In steganography, one is more concerned about the concealment of secret information as compared to encryption, where the objective is to render the secret message into an unreadable format. In certain situations, mere encryption of messages will attract undue attention.

The conventional techniques of image steganography use the pixels of images for concealing the secret information. One common practice is to embed the secret message by manipulating the Least Significant Bits

(LSBs). Though this practice is widely used, it is far from being secure. If the message is embedded in an easily readable form in the image, anyone who manages to extract the information can decode the original message without any difficulty.

The other major flaw in this method is the manner in which the data is embedded. Most basic steganographic systems embed the hidden data using a sequential method in which the data is placed into pixels from the first part of the image in a fixed order. While this technique may be easy to implement, it leaves the system open to steganalysis through the creation of patterns which could be detected statistically.

This highlights the difference between concealing the data and concealing it in a secure manner. The steganography algorithm should not rely solely on the image for the protection of its contents. In case an attacker is able to discover the hidden data, the information should still be secure through the use of cryptographic methods.

The goal of this project is to design a more secure system for image-based steganography, integrating encryption and unpredictable embedding of data to increase the security level. First of all, the secret message can be encoded using one of the cryptographic methods in order to prevent the receiver from obtaining the information directly from the payload. Thus, the encrypted data will be inserted into the image in a more sophisticated way compared to a regular sequence.

As such, this approach guarantees several levels of protection. The goal of steganography is to hide the very presence of the message. At the same time, encryption protects the confidentiality of the information regardless

of whether it was detected or not. This way, this project proves that there are ways of improving conventional steganographic methods to obtain better protection of information.

Thus, the current project is about inserting a secret message into an image and about overcoming certain security drawbacks that exist in steganographic systems.

---

Both these weaknesses in security can be overcome by ShadowCrypt using two different layers. Confidentiality is guaranteed through use of the AES-256-GCM algorithm for encryption along with the detection of any tampering of the message. Encryption of the message uses a key generated from the password of the user using the PBKDF2-HMAC-SHA256 algorithm. Concealment is done using the Password-scattered LSB matching method which involves scattering of the encrypted data in the image instead of storing it in sequential order.

---

### 3. Threat Model

The ShadowCrypt software employs the following threat model:

**Stego-image exposure:** It is possible that an attacker may have access to the PNG file containing the hidden message.

**Password secrecy:** The attacker is not aware of the password that was used to encrypt and embed the message.

**Message confidentiality:** In case the message is extracted by the attacker, the original message is secure due to the fact that it is encrypted with the **AES-256-GCM**.

**Statistical analysis:** The attacker can employ the standard statistical techniques such as chi-square and RS to detect any hidden information.

**Scattered embedding:** Using the password to seed the scattered LSB-matching technique is less predictable than the traditional sequential LSB replacement.

**Password as the security boundary:** In case the password is weak or guessed, the whole system fails to provide any security.

**Limited image integrity:** In case the PNG file is resaved, re-compressed, edited, etc., it may lead to data corruption.

**Limited scope of protection:** ShadowCrypt is not expected to provide protection against all the advanced methods of steganalysis, particularly those employing machine learning techniques.

---

## 4. Cryptography

In order to ensure the confidentiality and integrity of the hidden message, ShadowCrypt implements various state-of-the-art cryptographic algorithms. The cryptographic construction includes AES-256-GCM for encryption, PBKDF2-HMAC-SHA256 for key derivation, and generation of random salts and nonces for each encryption.

**Encryption – AES-256-GCM:** AES-256-GCM is used as the primary encryption algorithm. AES-256-GCM is confidentiality and authenticity algorithm which means that the encrypted message cannot be deciphered without the proper key and any changes made to the message will be detected during the decryption process.

**Key derivation – PBKDF2-HMAC-SHA256:** A user password is hashed by PBKDF2 with **200,000** iterations and **HMAC-SHA256** to generate **32-bytes long encryption key**. This approach is better than using the password or just its hash as the key for AES encryption since such a computation makes brute force attacks more expensive.

**Salt:** A **16-bytes long random salt** is generated for each encryption. This salt is stored together with the encrypted payload and is used in the process of key derivation. Since salts are generated randomly, the same password produces different keys for different encryptions.

**Nonce:** A unique 12-byte nonce is used for each encryption of AES-GCM. Nonce is necessary for the functioning of GCM algorithm and it should not be reused with the same encryption key.

The data is saved in the image by constructing the following format of payload:

**[4 bytes: ciphertext length][16 bytes: salt][12 bytes: nonce][ciphertext + authentication tag]**

This makes it possible for ShadowCrypt to get all the necessary cryptography parameters during decryption and check the integrity of the encrypted data.

---

## **5. Steganographic Design**

### **5.1 Mitigating the Flaw of Sequential Embedding**

In general, LSB based steganography methods embed the bits of a message one-by-one starting with the first pixel and then continuing with other pixels sequentially. While the method is simple, this leads to the formation of a predictable pattern of embedding. Only a specific region of the image gets affected, which may lead to the emergence of statistical differences between the affected and non-affected portions. Moreover, directly changing the value of the pixel in accordance with the message bit results in detectable frequency change of the pixel value pairs. These patterns can be detected using statistical steganalysis methods such as chi-square analysis.

### **5.2 Scattered Embedding Based on Passwords**

The problem is resolved in ShadowCrypt due to the use of scattered embedding order, which is generated pseudo-randomly.

The permutation will be based on a value that is calculated using SHA-256 hashing algorithm from the user's password. Therefore, the embedding order depends on the user's password but not the encryption key derivation process. This will make it impossible for any malicious attacker who does not have the password to know the positions at which the hidden information is stored.

The embedding order is deliberately selected based on the password and not the encryption salt which will be included in the encrypted payload. The salt is necessary for the decryption of the message as the embedding

order needs the salt in order to get the positions of the pixels. The salt will then need the embedding order in order to derive its positions in the image.

Through using pseudo-randomly placed pixels for the encrypted payload, ShadowCrypt escapes from the clear sequence of a basic LSB implementation approach.

### **5.3 LSB Matching Rather than LSB Substitution**

ShadowCrypt employs the technique of **LSB matching** rather than traditional LSB substitution to minimize the generation of statistical patterns during the process of data embedding. In simple LSB substitution, the value of a pixel is changed directly so that its least significant bit will correspond to the needed message bit. This will cause certain statistical changes in the distribution of pixel values.

With **LSB matching**, ShadowCrypt will first check if the least significant bit of the current pixel is equal to the needed bit. If so, no change is made to the pixel value. However, if the least significant bit does not match the needed bit, then the pixel value is incremented or decremented by 1, while remaining within the range of **0-255**.

Thus, LSB matching produces much less pronounced changes in the pixel values compared to simple substitution. This helps to avoid generating specific statistical artifacts which occur during LSB substitution.

## 6. Testing

### Img01: Running\_tool & CLI

```
PS D:\STEGNOGRAPHY> py stego.py

AES-256-GCM STEGO TOOL
encrypted * scattered LSB matching * password-derived ordering

1 Encrypt / Hide message
2 Decrypt / Extract message
3 Check image capacity
4 Exit

Choose: |
```

### Img02: Encrypting image

```
Choose: 1

Encrypt

Image file: test1.png
Output image name: test1_encptd.png
Message: any message //
Password:
Confirm password: |
```

### Img03: Encryption Success

```
D:\STEGNOGRAPHY\stego.py:68: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  pixels = list(image.getdata())
✓ Success. Output saved to test1_encptd.png

1 Encrypt / Hide message
2 Decrypt / Extract message
3 Check image capacity
4 Exit

Choose: |
```

### Img04: Decryption using password

```
Choose: 2

Decrypt

Stego image: test1_encptd.png
Password:
D:\STEGNOGRAPHY\stego.py:108: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  pixels = list(image.getdata())
any message // Hidden message

1 Encrypt / Hide message
2 Decrypt / Extract message
3 Check image capacity
4 Exit

Choose: |
```

#### Img05: Fail | Wrong Password

```
Choose: 2
Decrypt
Stego image: test1_encptd.png
Password:
Error: Image does not contain enough data.

1  Encrypt / Hide message
2  Decrypt / Extract message
3  Check image capacity
4  Exit
Choose: |
```

#### Img06: Image Capacity Checking

```
Choose: 3
Capacity check
Image file: test1.png

Property      Value
-----
Dimensions    1919 x 1078
Raw channel capacity  6,206,946 bits
Fixed overhead  32 bytes (salt+nonce+length)
Max message size  775,719 bytes (~775719 ASCII chars)

1  Encrypt / Hide message
2  Decrypt / Extract message
3  Check image capacity
4  Exit
Choose: |
```

---

## 7. Known Limitations

ShadowCrypt is created to enhance the security of simple image steganography, yet there are several limitations that users need to be aware of. Those limitations stem from the LSB technique and must be taken into account when working with the software.

1. **It is vulnerable to image manipulations:** Scaling, re-compression, filtering, or any other change can corrupt or completely erase the hidden message. That is because of the limitations of the LSB method and not something unique for ShadowCrypt.

2. **Image re-compression by messaging services:** Software like WhatsApp or Telegram might re-compress the images if they are sent as photos. This will erase the message. To protect the payload, the stego image needs to be transferred as an **\*\*original file or document** and not as a photo.

3. **Not invulnerable to detection:** Even though the scattered LSB matching algorithm was developed to decrease statistical anomalies that come with sequential LSB substitution, it is not protected from the attacks of machine learning steganalysis targeted against LSB techniques.

4. Password is the major security barrier: The detection of the existence of hidden data does not give away the original data since the data is encrypted through AES-256-GCM. But the use of a weak or predictable password is a huge risk for the security of the whole encrypted message. Hence, password strength is an important aspect of security in the process.

5. Only PNG outputs: ShadowCrypt creates PNG images because of lossy compression in JPEG format that changes pixel values and therefore destroys the hidden data. Hence, although the original data is in another format, the stego data will be in PNG format.

---

## 8. Possible Future Work

ShadowCrypt is an improvement over the basic LSB embedding approach in terms of increased security but there still are other things that could be implemented in order to further improve the system.

JPEG-compatible steganography: It is possible to implement the embedding into the DCT (Discrete Cosine Transform) domain. In that way the embedded information will withstand JPEG compression and thus more advanced approaches such as F5 or J-UNIWARD will be needed.

Automatic carrier image selection: It might be useful to implement an automated carrier image selection feature, in which the program will examine the input image(s) and determine whether it can serve as a carrier for the hidden data based on the dimensions of the images and their embedding capacity and the size of the payload.

Empirical steganalysis benchmarking: Other tests worth conducting will involve the benchmarking of the ShadowCrypt algorithm with regard to modern steganalysis tools such as StegExpose or zsteg and assessing detection rates on various payloads and carrier images.

---

## 9. Instructions for Use

1. Install dependencies: ``pip install -r requirements.txt``  
[ install rich, pillow (PIL), cryptography ]
  2. Run: ``python stego.py``
  3. From the menu:
    - **Option 1** to hide a message: provide the carrier image path, desired output filename, the message, and a password (entered twice to confirm).
    - **Option 2** to extract a message: provide the stego image path and the password used to hide it.
    - **Option 3** to check an image's capacity before encrypting.
  4. When sharing the resulting stego image, send it as a **\*\*file/document\*\***, not as a "photo," on any messaging platform — see Section 7.
- 

## 10. Conclusion

ShadowCrypt presents a multi-level strategy for the safe information hiding through encryption and image steganography. The AES-256-GCM is responsible for the main protection of the message and guarantees confidentiality and integrity regardless of whether the payload is revealed. The PBKDF2-HMAC-SHA256 algorithm is utilized to generate the key to encrypt the message.

The second layer of the program uses password-derived scattered embedding and LSB matching to avoid patterns that can be seen in conventional sequential LSB algorithms. However, this type of concealment is considered to be only a

secondary step in protecting the message and is not taken as the ultimate means of hiding sensitive information.

In general, the work shows that the combination of cryptography and steganography can guarantee better protection than using either of them alone. Moreover, the project keeps the real expectations concerning the security of LSB-based techniques.

---